

Convert Infix expression to Postfix
expression

Consider the infix expression $exp = "a+b*c+d"$ and the infix expression is scanned using the iterator i , which is initialized as $i = 0$.

1st Step: Here $i = 0$ and $exp[i] = 'a'$ i.e., an operand. So add this in the postfix expression. Therefore, **postfix = "a"**.



postfix = "a"

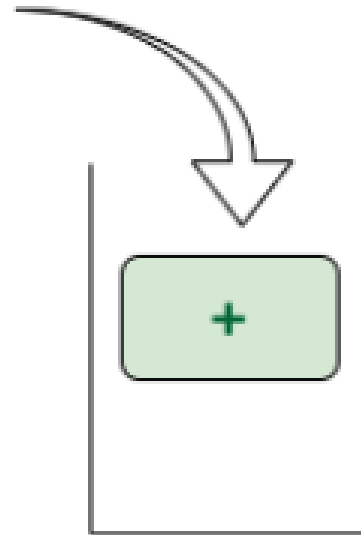
'a' is an operand. Add it in postfix expression

Add 'a' in the postfix

2nd Step: Here $i = 1$ and $exp[i] = '+'$ i.e., an operator. Push this into the stack. **postfix = "a"** and **stack = {+}**.

**Add '+' into
stack**

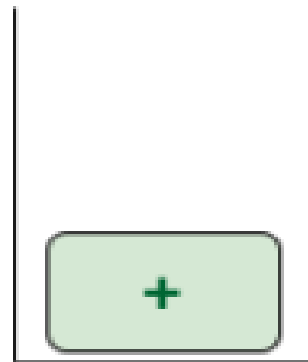
postfix = "a"



Stack is empty. Push '+' into stack

Push '+' in the stack

3rd Step: Now $i = 2$ and $exp[i] = 'b'$ i.e., an operand. So add this in the postfix expression. **postfix = "ab"** and **stack = {+}**.



postfix = "ab"

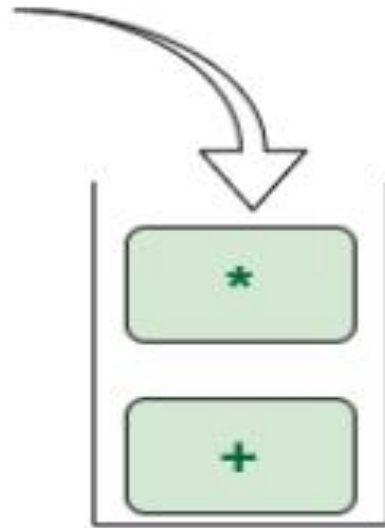
'b' is an operand. Add it in postfix expression

Add 'b' in the postfix

4th Step: Now $i = 3$ and $\text{exp}[i] = '*'$ i.e., an operator. Push this into the stack. **postfix = "ab"** and **stack = {+, *}**.

Add '*' into stack

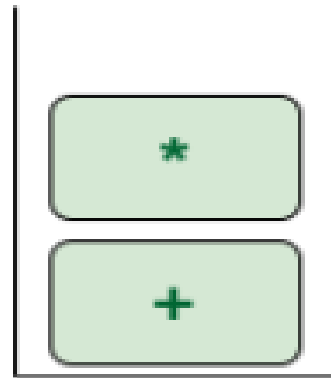
postfix = "ab"



*** has higher precedence. Push it into stack**

Push '' in the stack*

5th Step: Now $i = 4$ and $exp[i] = 'c'$ i.e., an operand. Add this in the postfix expression. **postfix = "abc"** and **stack = {+, *}**.



postfix = "abc"

'c' is an operand. Add it in postfix expression

Add 'c' in the postfix

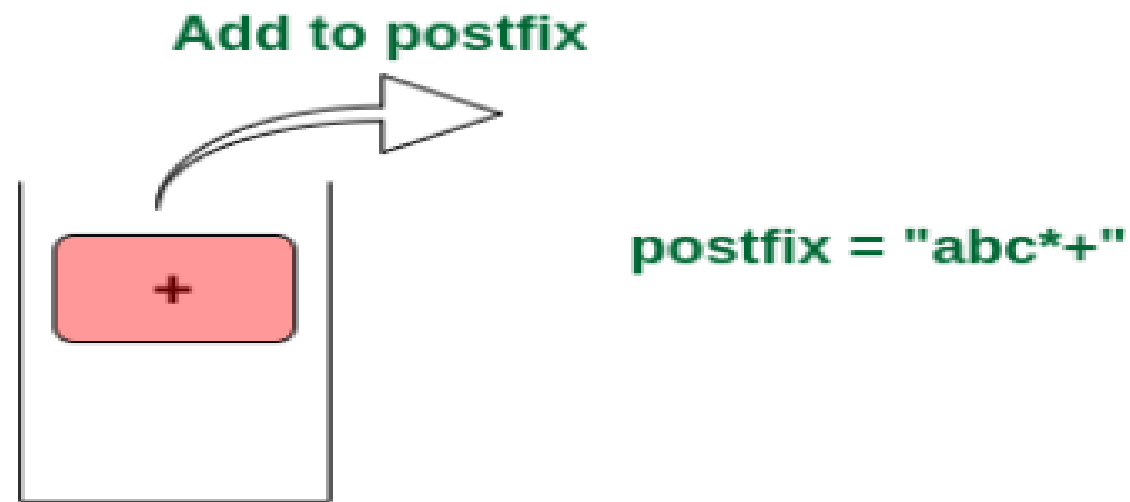
6th Step: Now $i = 5$ and $exp[i] = '+'$ i.e., an operator. The topmost element of the stack has higher precedence. So pop until the stack becomes empty or the top element has less precedence. '*' is popped and added in postfix. So **postfix = "abc*"** and **stack = {+}**.



stack top has higher precedence than +

Pop '' and add in postfix*

*Now top element is '+' that also doesn't have less precedence. Pop it.
postfix = "abc*+".*



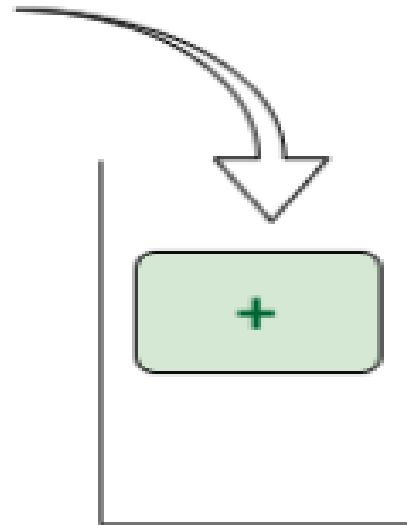
'+' and stack top has same precedence

Pop '+' and add it in postfix

*Now stack is empty. So push '+' in the stack. **stack = {+}**.*

**Add '+' into
stack**

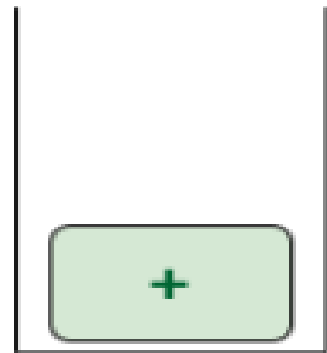
postfix = "abc*+"



Stack is empty. Push '+' into stack

Push '+' in the stack

*7th Step: Now $i = 6$ and $\text{exp}[i] = 'd'$ i.e., an operand. Add this in the postfix expression. **postfix = "abc*+d"**.*

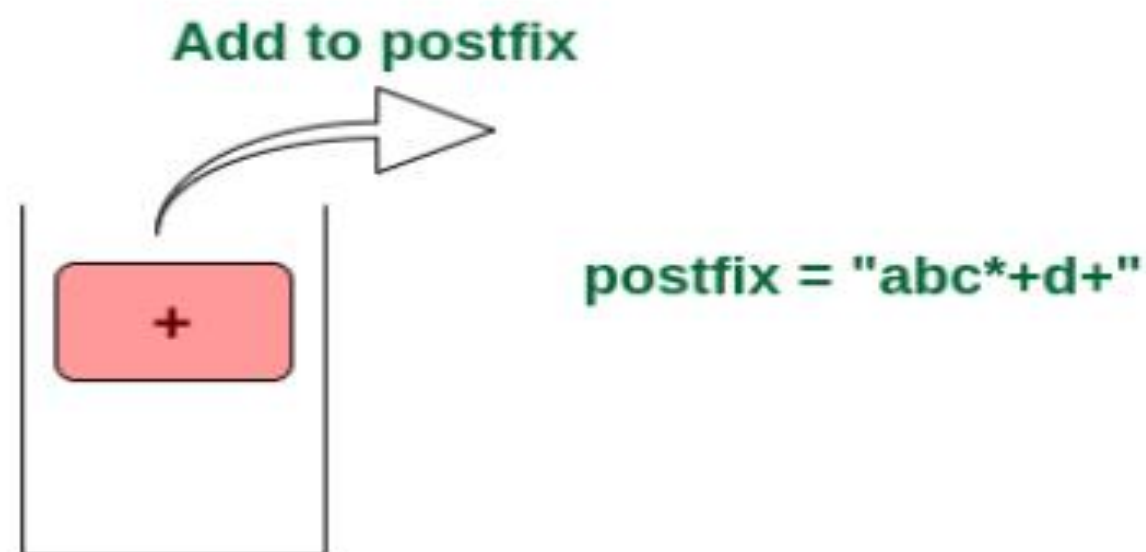


postfix = "abc*+d"

'd' is an operand. Add it in postfix expression

Add 'd' in the postfix

Final Step: Now no element is left. So empty the stack and add it in the postfix expression. **postfix = "abc*+d+"**.



Nothing left. So pop all operators

Pop '+' and add it in postfix

How to convert infix expression to prefix expression?

To convert an infix expression to a prefix expression, we can use the stack data structure. The idea is as follows:

- **Step 1:** Reverse the infix expression. Note while reversing each '(' will become ')' and each ')' becomes '('.
- **Step 2:** Convert the reversed infix expression to “nearly” postfix expression.
 - While converting to postfix expression, instead of using pop operation to pop operators with greater than or equal precedence, here we will only pop the operators from stack that have greater precedence.
- **Step 3:** Reverse the postfix expression.

The stack is used to convert infix expression to postfix form.

Convert Infix to Prefix Notation

Input

$A+(B*C)$

Infix Notation

1. Reverse given expression

$(C*B)+A$

Reverse Infix Notation

2. Convert to
Postfix Notation

$(CB*)A+$

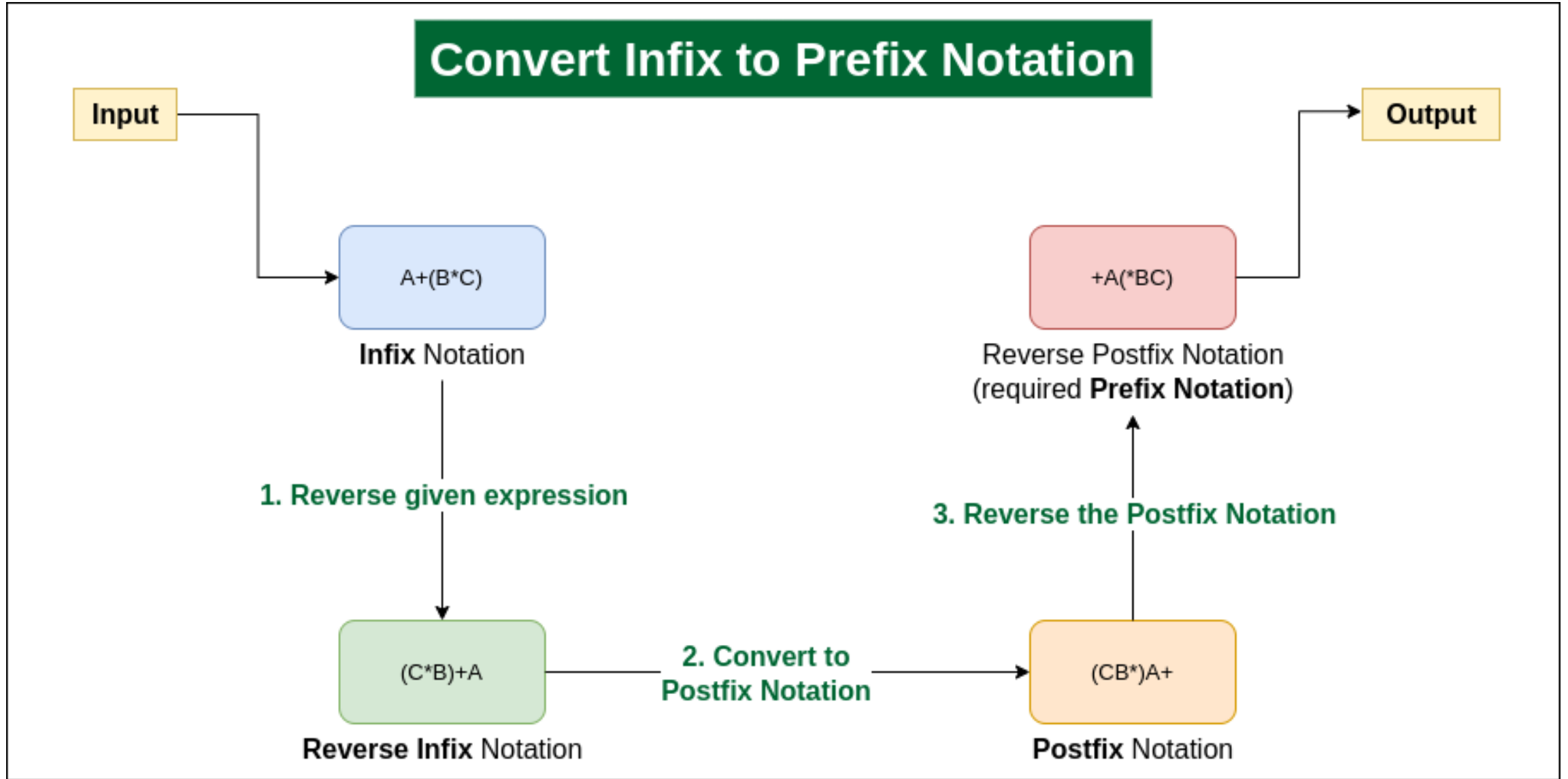
Postfix Notation

3. Reverse the Postfix Notation

$+A(*BC)$

Reverse Postfix Notation
(required **Prefix Notation**)

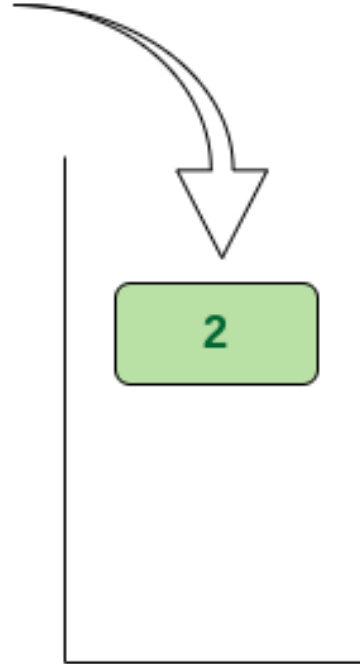
Output



Evaluation of Postfix Expression using Stack:

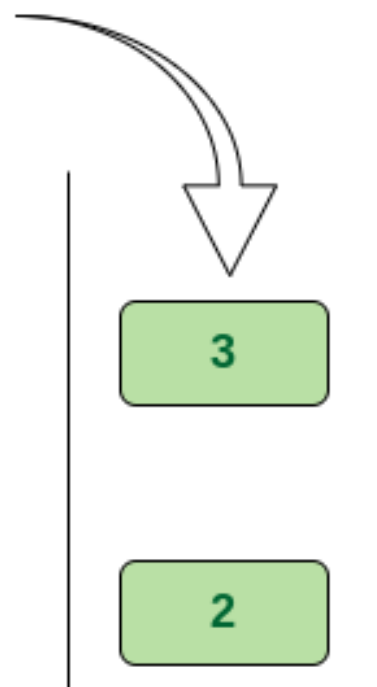
Consider the expression: $\text{exp} = "2\ 3\ 1\ *\ +\ 9\ -"$

Add 2 in the
stack



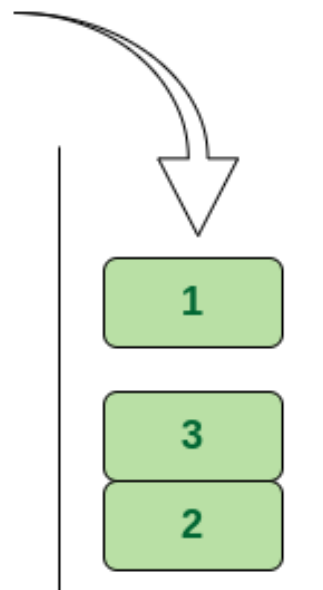
2 is an operand. Push it in stack

Add 3 in the
stack

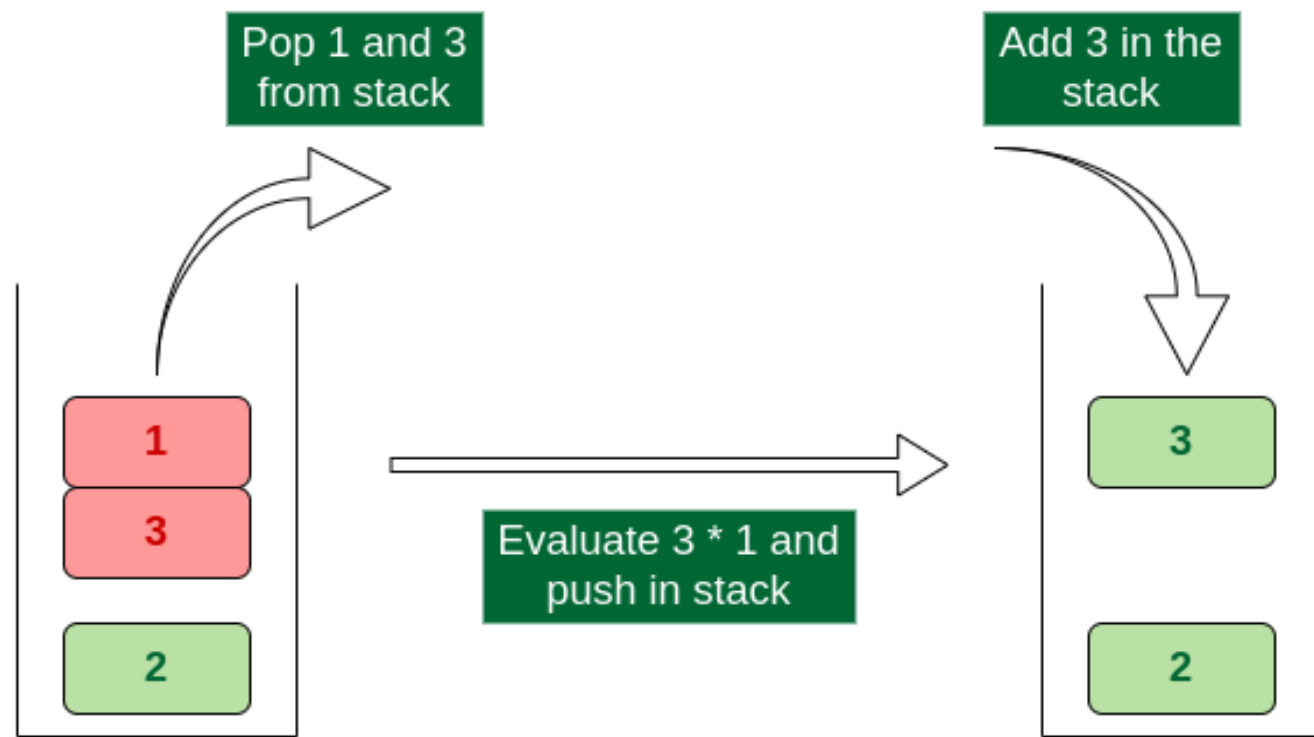


3 is an operand. Push it in stack

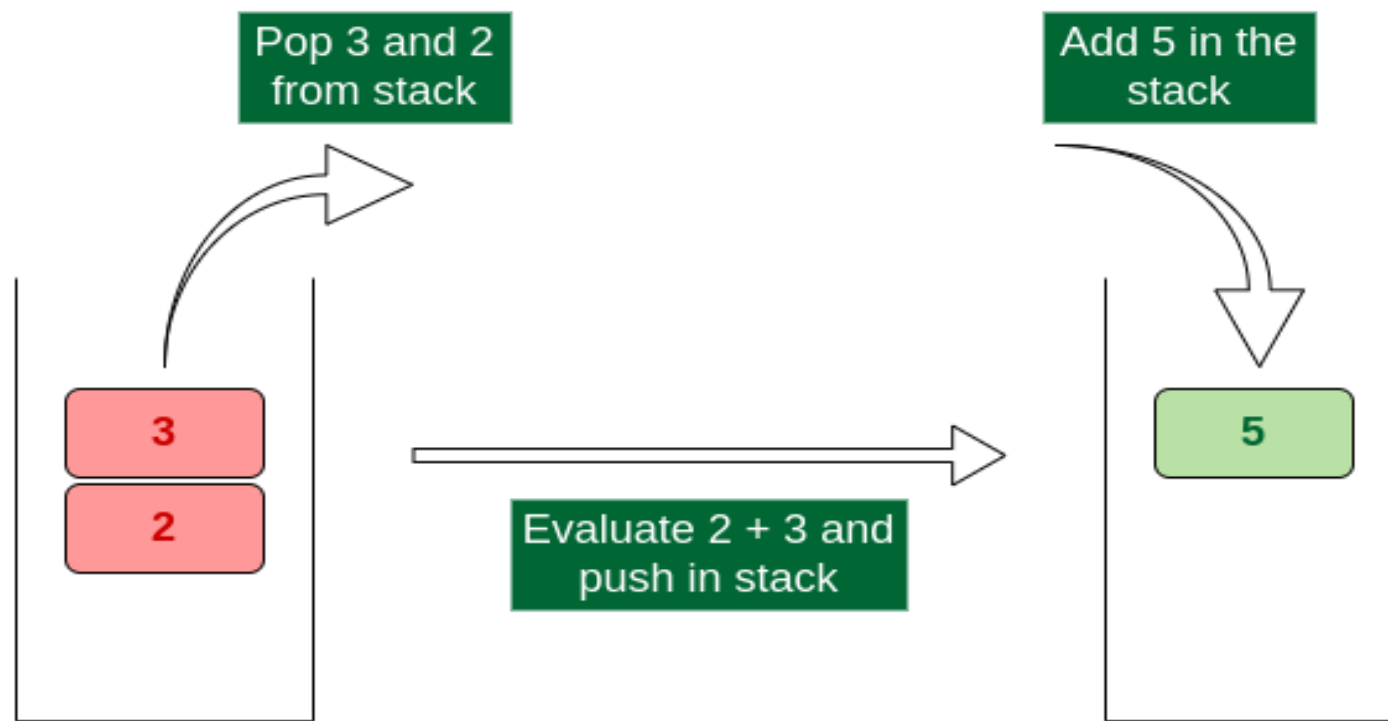
Add 1 in the
stack



1 is an operand. Push it in stack

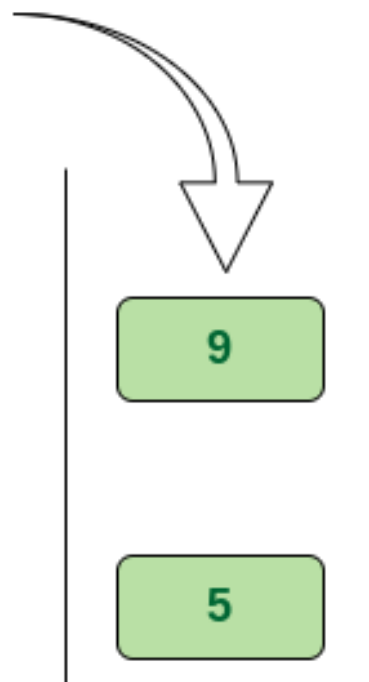


*** is an operator. Evaluate it and push result in stack**

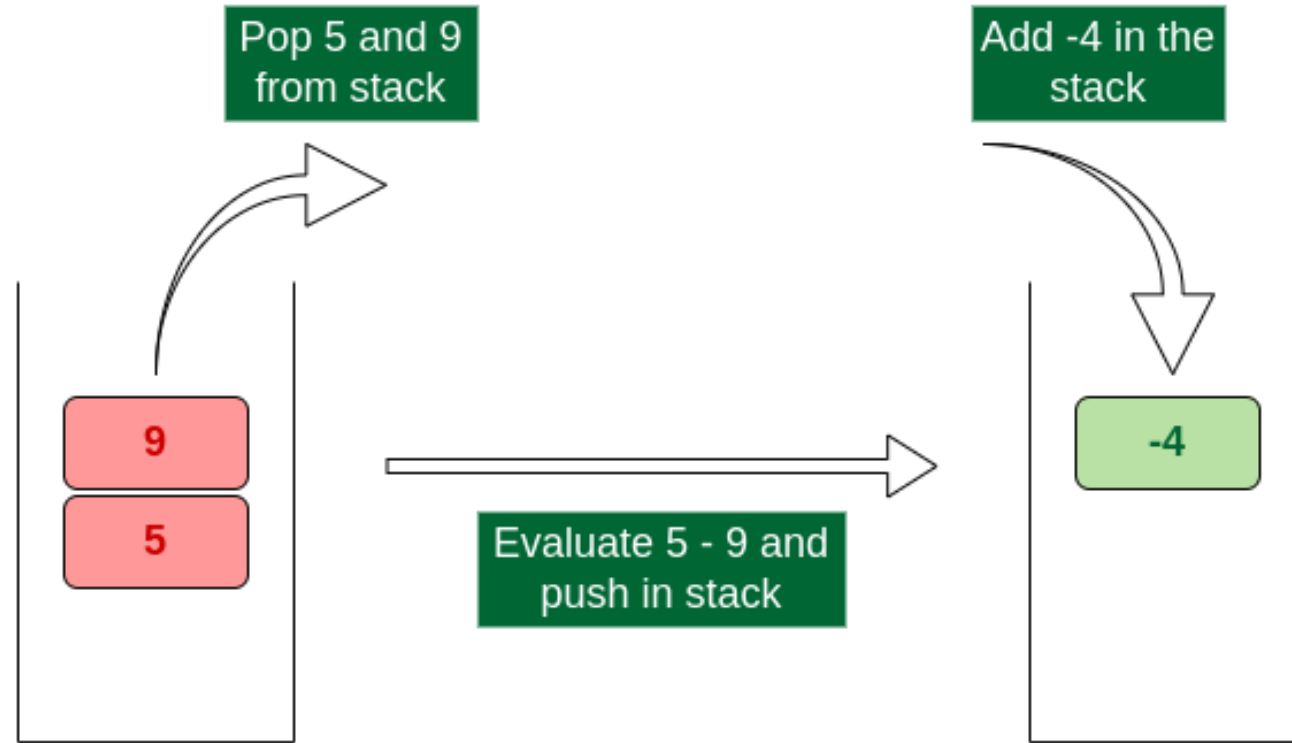


+ is an operator. Evaluate it and push result in stack

Add 9 in the
stack



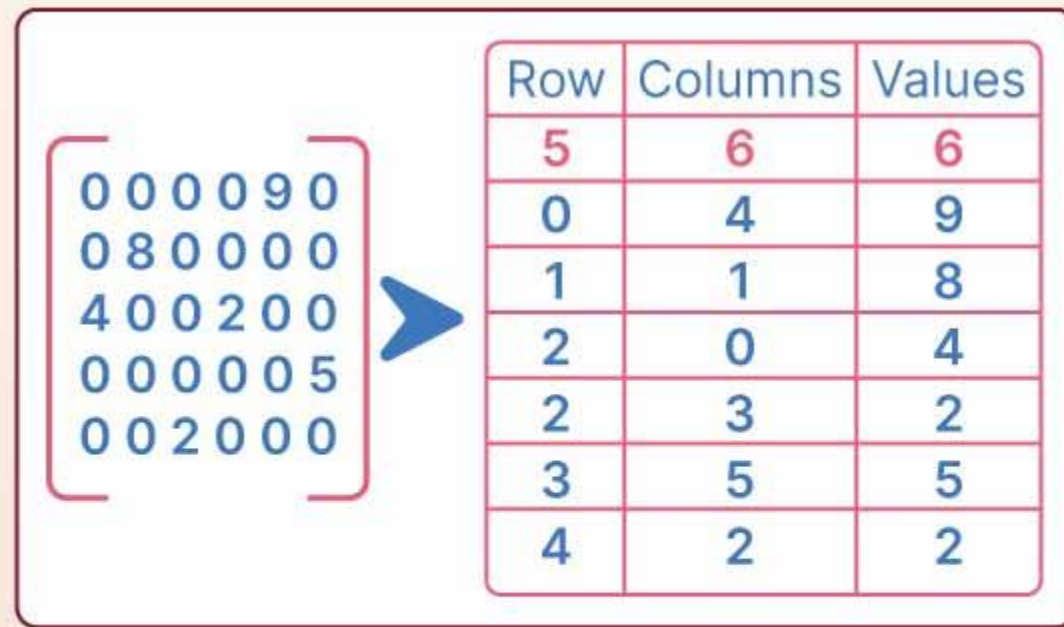
9 is an operand. Push it in stack



'-' is an operator. Evaluate it and push result in stack

Sparse Matrix

Representation & Operation



The diagram illustrates the conversion of a sparse matrix into a COO (Coordinate-Only) representation. On the left, a 6x6 matrix is shown with its non-zero elements highlighted in blue. A blue arrow points to a table on the right that lists the row, column, and value for each non-zero element.

Row	Columns	Values
5	6	6
0	4	9
1	1	8
2	0	4
2	3	2
3	5	5
4	2	2



Sparse Matrix



	0	1	2	3
0	0	4	0	5
1	0	0	3	6
2	0	0	2	0
3	2	0	0	0
4	1	0	0	0

Table Structure

Row	Column	Value
0	1	4
0	3	5
1	2	3
1	3	6
2	2	2
3	0	2
4	0	1
5	4	7

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 9 & 0 & 4 & 0 \\ 0 & 8 & 0 & 0 & 0 & 7 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

+

=

$$B = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 18 & 0 & 0 & 0 \\ 0 & 3 & 0 & 24 & 0 & 17 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$C = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 2 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 27 & 0 & 4 & 0 \\ 0 & 11 & 0 & 24 & 0 & 24 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 9 & 0 & 4 & 0 \\ 0 & 8 & 0 & 0 & 0 & 7 \\ 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

	0	1	2	3	4	5	6
A	5	1	2	2	3	3	4
	6	5	3	5	2	6	1
	6	3	9	4	8	7	2

$$B = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 18 & 0 & 0 & 0 \\ 0 & 3 & 0 & 24 & 0 & 17 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & 0 \end{bmatrix} \end{matrix}$$

	0	1	2	3	4	5	6
B	5	1	2	3	3	3	5
	6	1	3	2	4	6	4
	6	2	18	3	24	17	6

	0	1	2	3	4	5	6
A	5	1	2	2	3	3	4
	6	5	3	5	2	6	1
	6	3	9	4	8	7	2

	0	1	2	3	4	5	6
B	5	1	2	3	3	3	5
	6	1	3	2	4	6	4
	6	2	18	3	24	17	6

	0	1	2	3	4	5	6	7	8	9
C	5	1	1	2	2	3	3	3	4	5
	6	1	5	3	5	2	4	6	1	4
	9	2	3	27	4	11	24	24	2	6